Array Cycle Detection & Missing Numbers

O(1) Space Solutions Cheatsheet

LC 287: Find the Duplicate Number

Problem: Array of $n+1$ integers, each in $[1, n]$. Find the duplicate.
Constraint: Can't modify array, O(1) space
Example: $[1, 3, 4, 2, 2] \rightarrow 2$

Solution: Floyd's Cycle Detection

```java
public int findDuplicate(int[] nums) {
    // Phase 1: Find intersection point
    // Time: O(n), Space: O(1)
    int slow = nums[0];
    int fast = nums[0];

    do {
        slow = nums[slow];          // Move 1 step: O(1)
        fast = nums[nums[fast]];    // Move 2 steps: O(1)
    } while (slow != fast);

    // Phase 2: Find cycle entrance (duplicate)
    // Time: O(n), Space: O(1)
    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];          // O(1)
        fast = nums[fast];          // O(1)
    }

    return slow; // Overall: O(n) time, O(1) space
}
```

Dry Run: $nums = [1, 3, 4, 2, 2]$

```
Index:  0  1  2  3  4
Value: [1, 3, 4, 2, 2]
        |  |  |  | /
        v  v  v  vv
       [1]->[3]->[2]<->[4]
                 ^------|

Phase 1: Find intersection
slow: 0->1->3->2->4->2
fast: 0->3->4->2->2
Intersection at index 2 (value=4)

Phase 2: Find entrance
slow: 0->1->3->2
fast: 2->4->2
Meet at index 2 (value=2)
```

Tips & Tricks

- **Why it works:** Array is treated as linked list where $nums[i]$ points to index $nums[i]$
- **Cycle exists because:** Duplicate creates two pointers to same node
- **Distance property:** d(start to entrance) = d(intersection to entrance)
- **Pattern:** Same as LC 142 (Linked List Cycle II)

Common Mistakes

- Starting fast/slow at index 0 (start at nums[0])
- Using `while` instead of `do-while` for phase 1
- Resetting `fast` to 0 in phase 2 (only reset `slow`)
- Moving fast 1 step in phase 2 (both move 1 step)

Complexity

Time: O(n) - Two linear passes
Space: O(1) - Only two pointers
Best for: Can't modify array

LC 141: Linked List Cycle

Problem: Detect if linked list has a cycle
Example: $1 \rightarrow 2 \rightarrow 3 \rightarrow 2$ (cycle) $\rightarrow$ true

```java
public boolean hasCycle(ListNode head) {
    // Time: O(n), Space: O(1)
    if (head == null) return false; // O(1)

    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;       // O(1) per iteration
        fast = fast.next.next;  // O(1) per iteration

        if (slow == fast) {     // O(1) comparison
            return true;
        }
    }
    return false;
    // Total: O(n) time, O(1) space
}
```

Visualization:

```
No Cycle:
1 -> 2 -> 3 -> 4 -> null
s         f
     s              f (exits)

With Cycle:
1 -> 2 -> 3 -> 4
     ^---------|
s    f
     s         f
          s    f
          sf (collision!)
```

Tips & Tricks

- **Tortoise & Hare:** Slow moves 1, fast moves 2
- **Meeting guaranteed:** In cycle, they close gap by 1 each step
- **Null check:** Check both `fast` and `fast.next`

LC 142: Linked List Cycle II

Problem: Return the node where cycle begins
Example: $1 \rightarrow 2 \rightarrow 3 \rightarrow 2 \rightarrow$ node 2

```java
public ListNode detectCycle(ListNode head) {
    // Time: O(n), Space: O(1)
    if (head == null) return null;

    ListNode slow = head, fast = head;

```

```java
    // Phase 1: Detect cycle - O(n)
    while (fast != null && fast.next != null) {
        slow = slow.next;        // O(1)
        fast = fast.next.next;   // O(1)

        if (slow == fast) {
            // Phase 2: Find entrance - O(n)
            slow = head;
            while (slow != fast) {
                slow = slow.next; // O(1)
                fast = fast.next; // O(1)
            }
            return slow;
        }
    }
    return null;
    // Total: O(n) time, O(1) space
}
```

Mathematical Proof:

```
Let:
- L = distance from head to cycle start
- C = cycle length
- k = distance from start to meeting point

When they meet:
slow traveled: L + k
fast traveled: L + k + nC (n loops)

Since fast is 2x speed:
2(L + k) = L + k + nC
L + k = nC
L = nC - k

Therefore: Distance from head to entrance
        = Distance from meeting to entrance
```

LC 645: Set Mismatch

Problem: Array $[1, n]$ with one duplicate, one missing
Example: $[1, 2, 2, 4] \rightarrow [2, 3]$

```java
public int[] findErrorNums(int[] nums) {
    // Time: O(n), Space: O(1)
    int duplicate = -1, missing = -1;

    // Phase 1: Mark visited - O(n)
    for (int i = 0; i < nums.length; i++) {
        int index = Math.abs(nums[i]) - 1; // O(1)

        if (nums[index] < 0) {        // O(1) check
            duplicate = index + 1;    // Found
    duplicate
        } else {
            nums[index] = -nums[index]; // Mark
    visited: O(1)
        }
    }

    // Phase 2: Find missing - O(n)
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] > 0) {           // O(1) check
            missing = i + 1;
            break;
        }
    }
    return new int[]{duplicate, missing};
    // Total: O(n) time, O(1) space
}
```

Detailed Dry Run: $[1, 2, 2, 4]$

```
Index:  0   1   2   3
Value: [1,  2,  2,  4]

i=0: val=1, idx=0
  nums[0]=1 (positive)
  Mark: [-1, 2, 2, 4]

i=1: val=2, idx=1
  nums[1]=2 (positive)
  Mark: [-1, -2, 2, 4]

i=2: val=2, idx=1
  nums[1]=-2 (NEGATIVE!)
  duplicate = 1+1 = 2
  Array: [-1, -2, 2, 4]

i=3: val=4, idx=3
  nums[3]=4 (positive)
  Mark: [-1, -2, 2, -4]

Phase 2: Find positive
Final: [-1, -2, 2, -4]
            idx 2 is positive
            missing = 2+1 = 3
```

Tips & Tricks

- **Index as HashMap:** Value v maps to index $v - 1$
- **Sign as flag:** Negative = visited, Positive = not visited
- **Math.abs():** Needed because values get negated during processing
- **Mapping:** value $\xrightarrow{-1}$ index $\xrightarrow{+1}$ value

Common Mistakes

- Forgetting `Math.abs()` when calculating index
- Using `nums[i]` instead of `Math.abs(nums[i])`
- Off-by-one: Using `index = nums[i]` instead of `nums[i]-1`
- Returning `[missing, duplicate]` instead of `[duplicate, missing]`

LC 268: Missing Number

Problem: Array of n integers from $[0, n]$, find missing
Example: $[3, 0, 1] \rightarrow 2$

Solution 1: XOR (Best) ✅

```java
public int missingNumber(int[] nums) {
    // Time: O(n), Space: O(1)
    int xor = nums.length; // Start with n: O(1)

    for (int i = 0; i < nums.length; i++) {
        xor ^= i ^ nums[i];  // XOR both: O(1) per iteration
    }
    return xor;
    // Total: O(n) time, O(1) space
}
```

XOR Properties:

```
a ^ a = 0
a ^ 0 = a
Commutative: a ^ b = b ^ a

Example: [3, 0, 1]
xor = 3 ^ (0^3) ^ (1^0) ^ (2^1)
    = 3 ^ 3 ^ 0 ^ 0 ^ 1 ^ 1 ^ 2
    = 0 ^ 0 ^ 0 ^ 2
    = 2
```

Solution 2: Sum Formula

```java
public int missingNumber(int[] nums) {
    // Time: O(n), Space: O(1)
    int n = nums.length;
    int expectedSum = n * (n + 1) / 2; // O(1)
    int actualSum = 0;

    for (int num : nums) {
        actualSum += num;  // O(1) per iteration
    }
    return expectedSum - actualSum;
    // Total: O(n) time, O(1) space
}
```

Tips & Tricks

- **XOR vs Sum:** XOR avoids overflow, Sum is more intuitive
- **Why XOR works:** All numbers cancel except missing one
- **Sum formula:** $\sum_{i=0}^{n} i = \frac{n(n+1)}{2}$

LC 442: Find All Duplicates in Array

Problem: Array where elements appear once or twice, find all duplicates
Example: $[4, 3, 2, 7, 8, 2, 3, 1] \rightarrow [2, 3]$

```java
public List<Integer> findDuplicates(int[] nums) {
    // Time: O(n), Space: O(1) (output doesn't count)
    List<Integer> result = new ArrayList<>();

    for (int i = 0; i < nums.length; i++) {
        int index = Math.abs(nums[i]) - 1; // O(1)

        if (nums[index] < 0) {      // O(1) check found
            result.add(index + 1); // Duplicate
        } else {
            nums[index] = -nums[index]; // Mark: O(1)
        }
    }
    return result;
    // Total: O(n) time, O(1) space
}
```

LC 448: Find All Numbers Disappeared in Array

Problem: Array $[1, n]$, find all numbers that don't appear
Example: $[4, 3, 2, 7, 8, 2, 3, 1] \rightarrow [5, 6]$

```java
public List<Integer> findDisappearedNumbers(int[] nums) {
    // Time: O(n), Space: O(1)

    // Phase 1: Mark present - O(n)
    for (int i = 0; i < nums.length; i++) {
        int index = Math.abs(nums[i]) - 1; // O(1)
        nums[index] = -Math.abs(nums[index]); // O(1)
    }

    List<Integer> result = new ArrayList<>();

    // Phase 2: Collect missing - O(n)
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] > 0) {          // O(1) check
            result.add(i + 1);
        }
    }
    return result;
    // Total: O(n) time, O(1) space
}
```

Dry Run: $[4, 3, 2, 7, 8, 2, 3, 1]$

```
Phase 1: Mark present
i=0: 4 -> idx=3 -> [4,3,2,-7,8,2,3,1]
i=1: 3 -> idx=2 -> [4,3,-2,-7,8,2,3,1]
i=2: |-2| -> idx=1 -> [4,-3,-2,-7,8,2,3,1]
i=3: |-7| -> idx=6 -> [4,-3,-2,-7,8,2,-3,1]
i=4: 8 -> idx=7 -> [4,-3,-2,-7,8,2,-3,-1]
i=5: |2| -> idx=1 -> (already negative)
i=6: |-3| -> idx=2 -> (already negative)
i=7: |-1| -> idx=0 -> [-4,-3,-2,-7,8,2,-3,-1]

Phase 2: Find positive
Final: [-4,-3,-2,-7,8,2,-3,-1]
                      ^ ^

               idx 4,5 positive
Result: [5, 6]
```

Tips & Tricks

- **Use `-Math.abs()`:** Handles duplicate values
- **Pattern:** Mark present, find absent
- **Inverse of:** Find All Duplicates

LC 41: First Missing Positive 👀

Problem: Find smallest missing positive integer
Example: $[3, 4, -1, 1] \rightarrow 2$

```java
public int firstMissingPositive(int[] nums) {
```

```java
    // Time: O(n), Space: O(1)
    int n = nums.length;

    // Phase 1: Cyclic sort - O(n)
    int i = 0;
    while (i < n) {
        int correctIdx = nums[i] - 1;
        // Place nums[i] at index nums[i]-1
        if (nums[i] > 0 && nums[i] <= n
            && nums[i] != nums[correctIdx]) {
            swap(nums, i, correctIdx); // O(1)
        } else {
            i++;
        }
    }

    // Phase 2: Find first mismatch - O(n)
    for (i = 0; i < n; i++) {
        if (nums[i] != i + 1) {    // O(1) check
            return i + 1;
        }
    }
    return n + 1;
    // Total: O(n) time, O(1) space
}

private void swap(int[] nums, int i, int j) {
    int temp = nums[i];  // O(1)
    nums[i] = nums[j];
    nums[j] = temp;
}
```

Cyclic Sort Visualization:

```
Input: [3, 4, -1, 1]
Goal:  [1, 2,  3, 4] or [1, ?, ?, ?]

i=0: nums[0]=3, should be at idx=2
  Swap 0<->2: [-1, 4, 3, 1]

i=0: nums[0]=-1, negative, skip
  i++: [-1, 4, 3, 1]

i=1: nums[1]=4, should be at idx=3
  Swap 1<->3: [-1, 1, 3, 4]

i=1: nums[1]=1, should be at idx=0
  Swap 0<->1: [1, -1, 3, 4]

i=1: nums[1]=-1, negative, skip
  i++: [1, -1, 3, 4]

i=2: nums[2]=3, at correct position
  i++: [1, -1, 3, 4]

i=3: nums[3]=4, at correct position
  Done: [1, -1, 3, 4]

Phase 2: Find mismatch
i=0: nums[0]=1 == 1
i=1: nums[1]=-1 != 2
Return: 2
```

Tips & Tricks

- **Cyclic Sort:** Place each element at its correct index
- **Key idea:** Value k belongs at index $k - 1$
- **Ignore:** Numbers ≤ 0 or $> n$ (can't fit)
- **While loop:** Keep swapping until current position is correct

Common Mistakes

- Using `for` instead of `while` (may need multiple swaps at one position)
- Not checking `nums[i] != nums[correctIdx]` (infinite loop with duplicates)
- Incrementing i after every swap (should only increment when position is correct)

Pattern Comparison

When to Use Each Pattern

Pattern	Use When	Problems
Floyd's Cycle	Can't modify array, find single duplicate	LC 287, 141, 142
Index Marking	Can modify, mark visited by negating	LC 442, 448, 645
XOR/Sum	Find missing/extra in known range	LC 268
Cyclic Sort	Place elements at correct positions	LC 41, 268

Complexity Summary

Time & Space Analysis

Problem	Time	Space	Modifies
Find Duplicate (Floyd's)	O(n)	O(1)	No
Set Mismatch (Marking)	O(n)	O(1)	Yes
Missing Number (XOR)	O(n)	O(1)	No
Missing Number (Sum)	O(n)	O(1)	No
First Missing Positive	O(n)	O(1)	Yes
Find All Duplicates	O(n)	O(1)	Yes
Find Disappeared Numbers	O(n)	O(1)	Yes

Key Insights & Tricks

Mental Models

1. Array as Implicit Linked List

```
Index:  0  1  2  3  4
Value: [1, 3, 4, 2, 2]
        |  |  |  | /
        v  v  v  vv
Think: 0->1->3->2->4->2 (cycle!)
```

2. Index as HashMap (Sign Marking)

```
Mapping: value v -> index (v-1)
Flag:    negative = visited
         positive = not visited

[4, 3, 2, 1] -> Process 4
 Mark index 3 -> [4, 3, 2, -1]
```

3. XOR Magic

```
Property: a ^ a = 0, a ^ 0 = a
Strategy: XOR all indices & values
          Pairs cancel, missing remains

[0,1,3] n=3
0^1^3 ^ 0^1^2^3 = 2
```

4. Cyclic Sort Intuition

```
Goal: Put each number at its "home"
      Number k -> Index k-1

[3, 4, -1, 1]
      |     |
      v     v
[1, ?, 3, 4]  <- Perfect!
Missing at index 1 -> Answer: 2
```

Interview Strategy

Step 1: Clarify Constraints
- Can I modify the array?
- What's the range of numbers?
- Are there duplicates? How many?
- Space complexity requirement?

Step 2: Choose Pattern
- Can't modify + single duplicate → Floyd's
- Can modify + mark visited → Index marking
- Find missing in $[0, n]$ → XOR/Sum
- Place at correct index → Cyclic sort

Step 3: Explain Approach
- "I'll treat the array as an implicit linked list..."
- "Using the sign bit as a visited flag..."
- "XOR will cancel out all pairs..."
- "Cyclic sort places each element at its correct position..."

Step 4: Handle Edge Cases
- Empty array / null
- Single element
- All duplicates
- No missing number

Debug Checklist

Floyd's Cycle:
☐ Used do-while for phase 1?
☐ Only reset slow in phase 2?
☐ Both move 1 step in phase 2?
☐ Started from nums[0], not index 0?

Index Marking:
☐ Used Math.abs() when calculating index?
☐ Correct mapping: value-1 for index?
☐ Checked if already negative before marking?
☐ Used -Math.abs() for disappeared numbers?

Cyclic Sort:
☐ Used while instead of for?
☐ Checked nums[i] != nums[correctIdx]?
☐ Only increment when position correct?
☐ Validated range: $1 \leq$ nums[i] $\leq$ n?

Created with LaTeX for Technical Interview Preparation
Practice on LeetCode: 287, 141, 142, 268, 442, 448, 645, 41